Feature matching

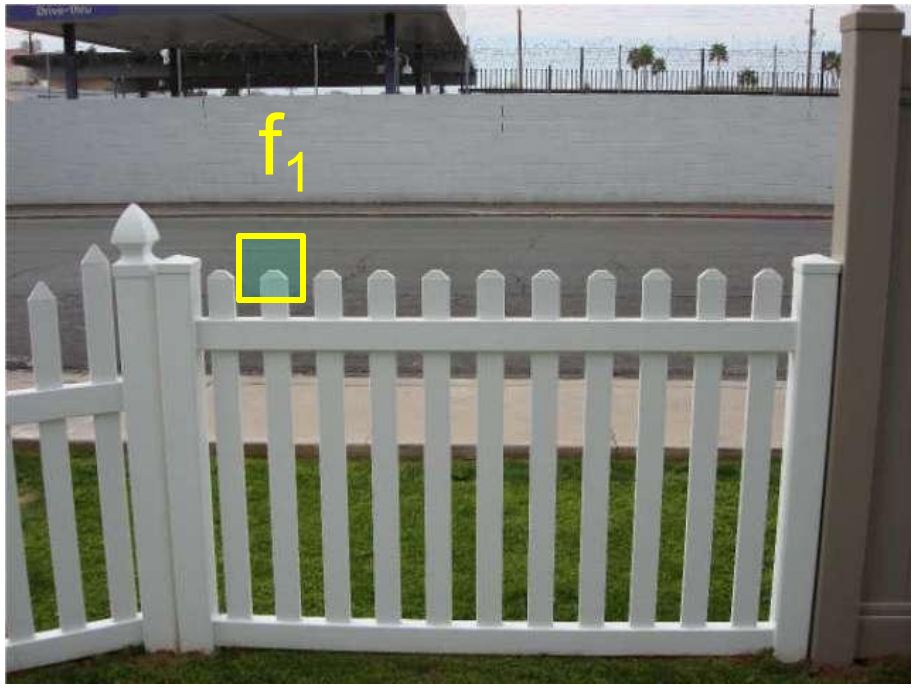
Given a feature in I_1, how to find the best match in I_2?

1. Define distance function that compares two descriptors
2. Test all the features in I_2, find the one with min distance
   (can be accelerated with a nearest neighbors search data structure, like a *kd*-tree)

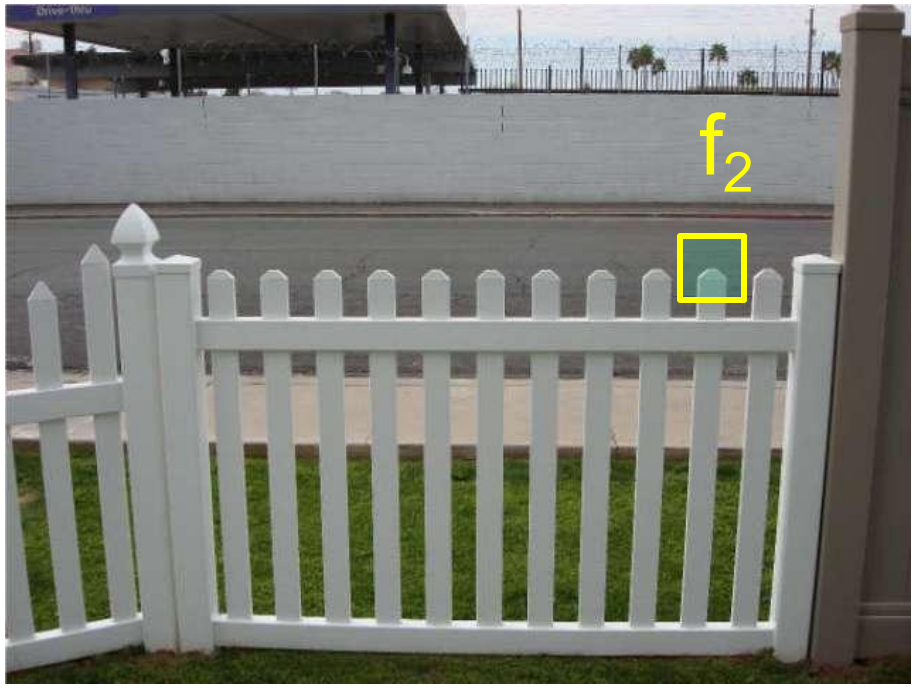
Feature distance

How to define the difference between two features f_1, f_2?

- Simple approach: L_2 distance, $\| f_1 - f_2 \|$
- can give small distances for ambiguous (incorrect) matches



I_1

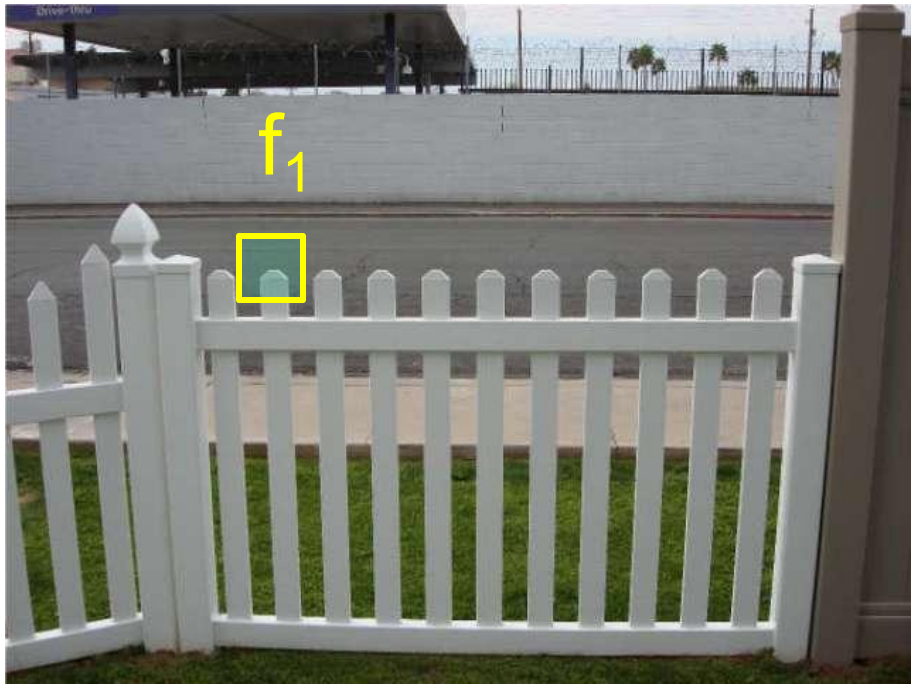


I_2

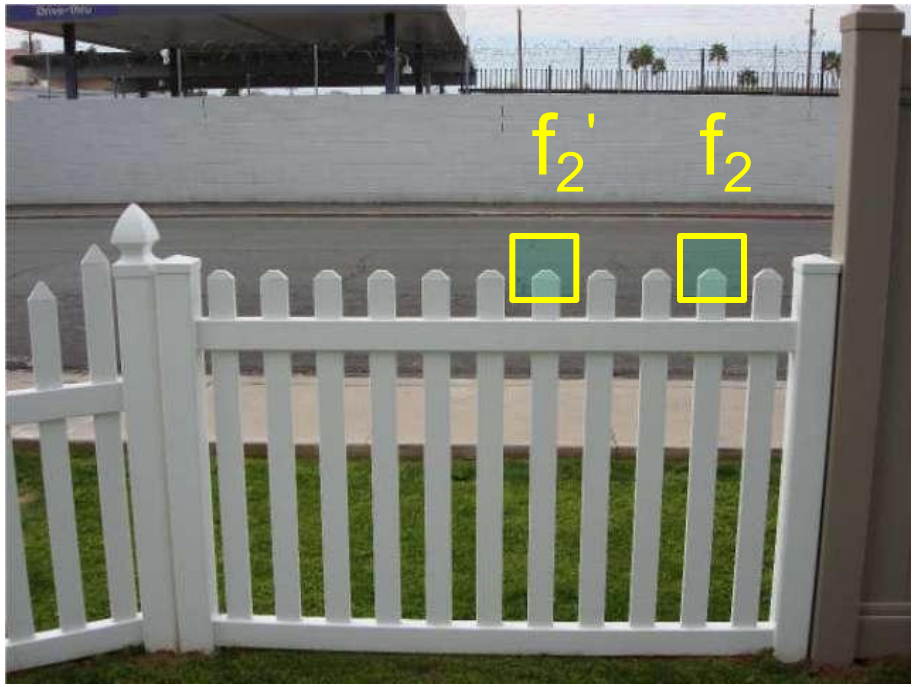
Feature distance

How to define the difference between two features f_1, f_2?

- Better approach: ratio distance = $\| f_1 - f_2 \| / \| f_1 - f_2' \|$
  - f_2 is the best SSD match to f_1 in I_2
  - f_2' is the 2nd best SSD match to f_1 in I_2
  - gives large values for ambiguous matches



I_1

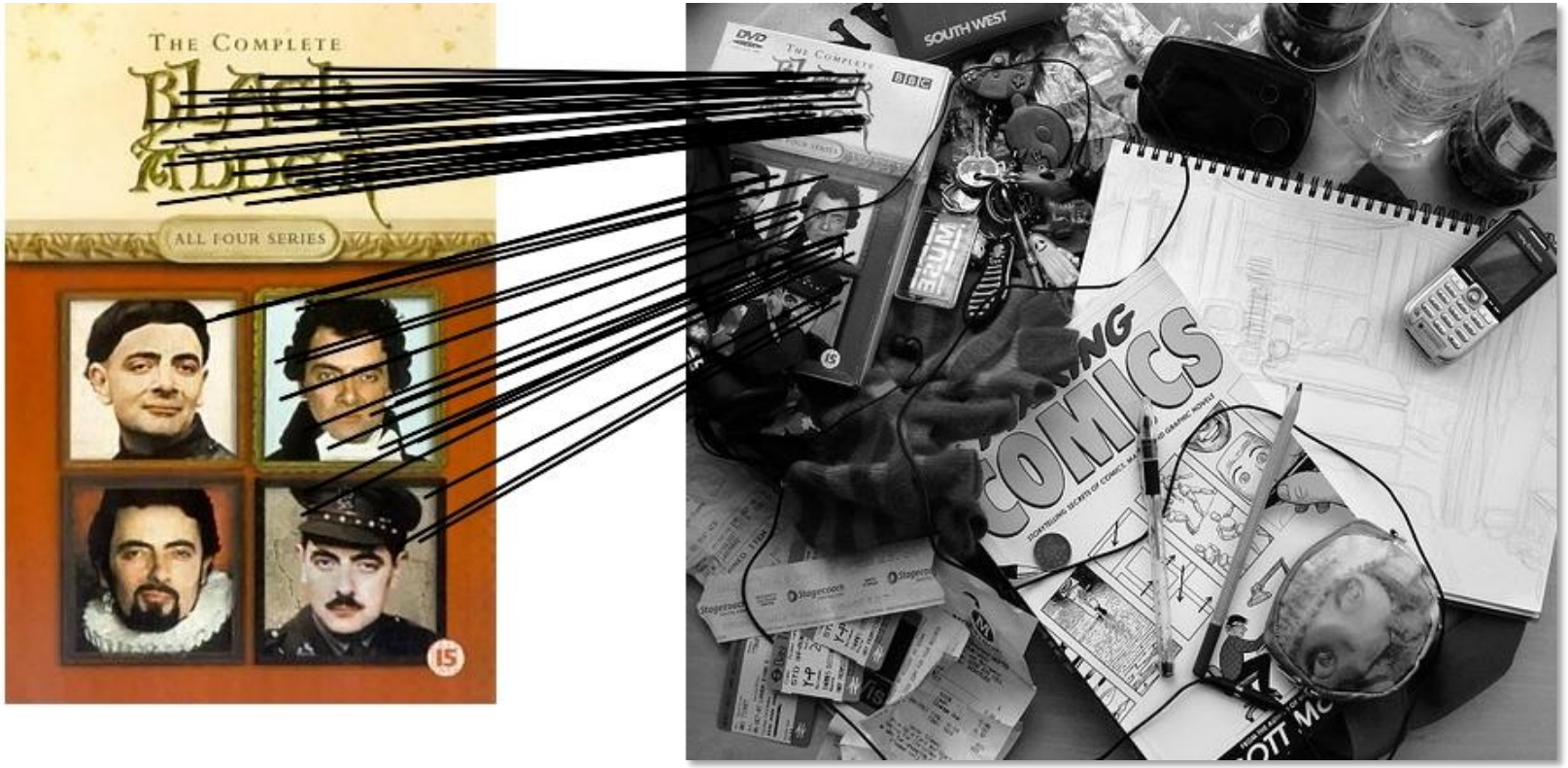


I_2

Feature distance

- Does the SSD vs "ratio distance" change the best match to a given feature in image 1?
- No, but it changes the distance, and it can change the ordering of matches from good to bad
- After we compute a set of matches, we *threshold* by distance (that is, throw out matches with distance > threshold)

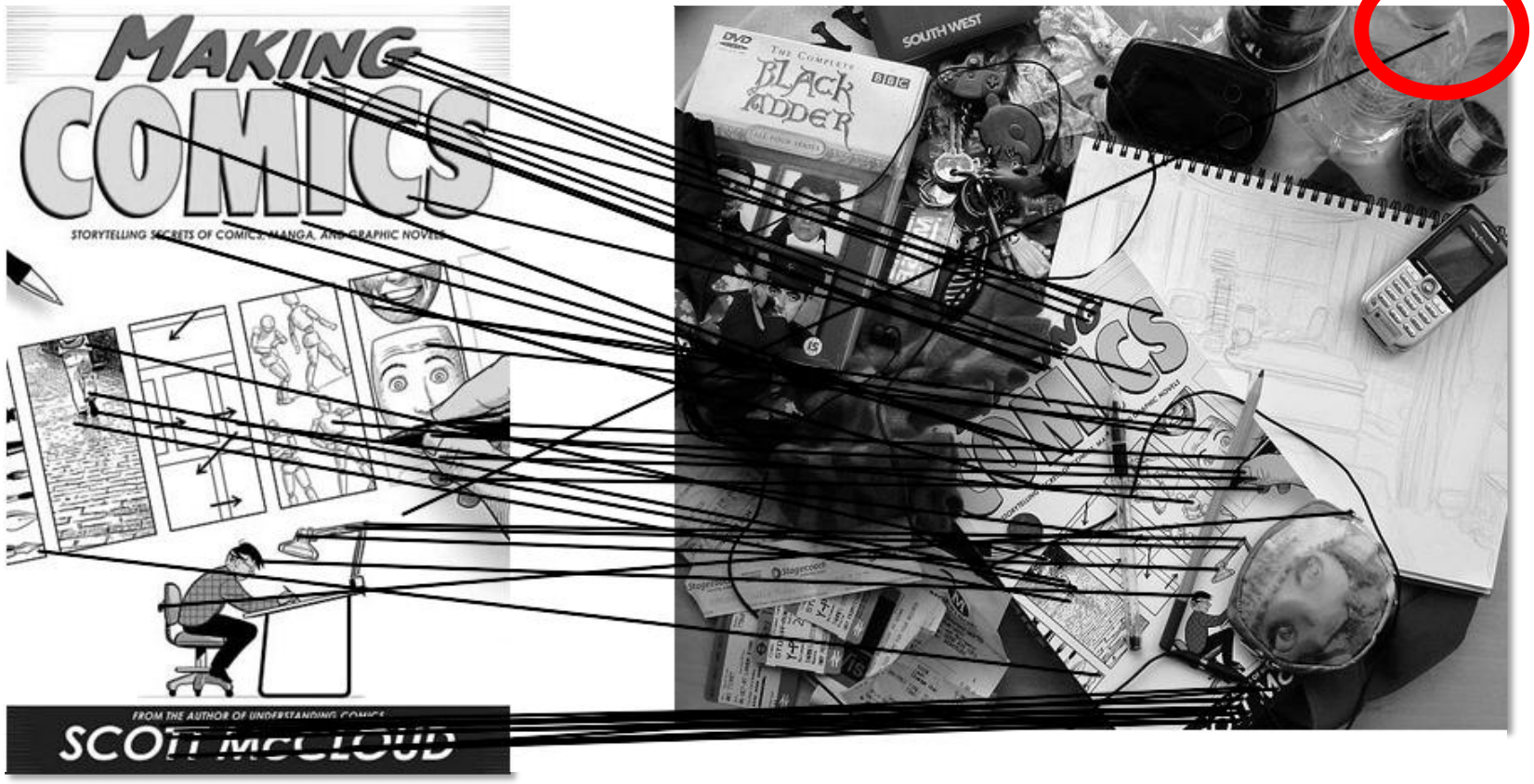
Feature matching example



58 matches (thresholded by ratio score)

Feature matching example

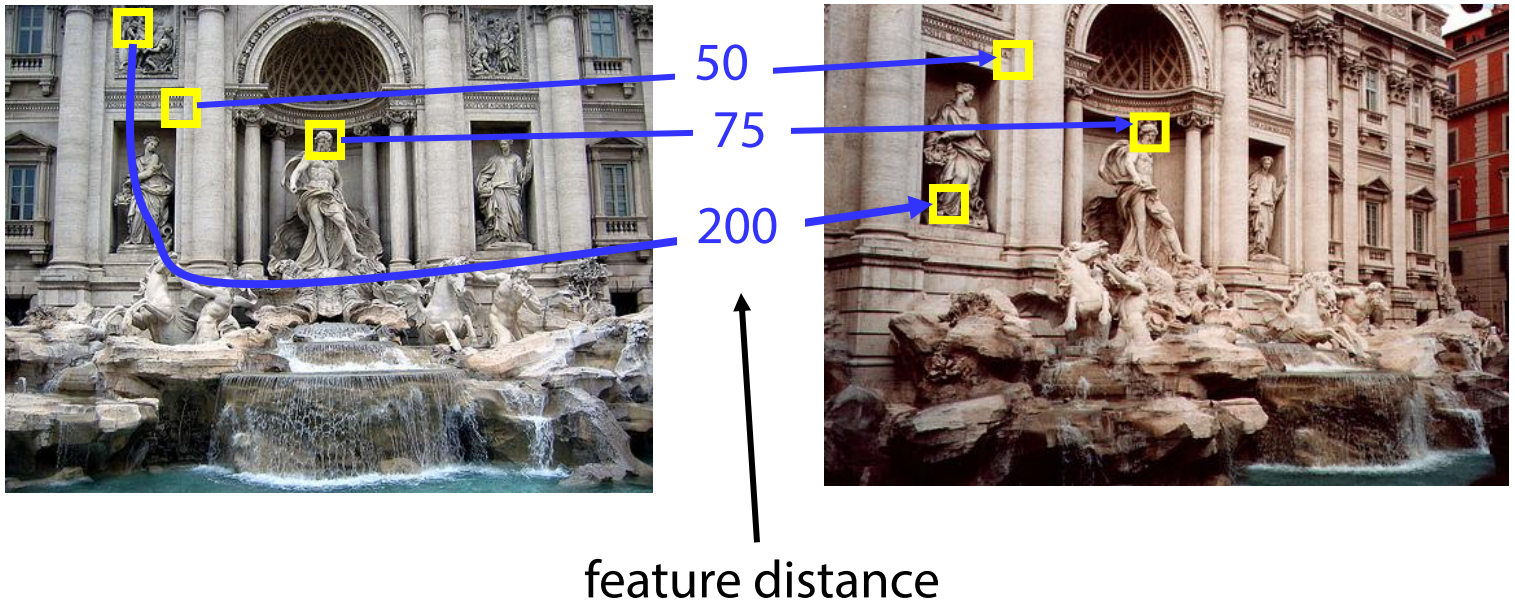
We'll deal with **outliers** later



51 matches (thresholded by ratio score)

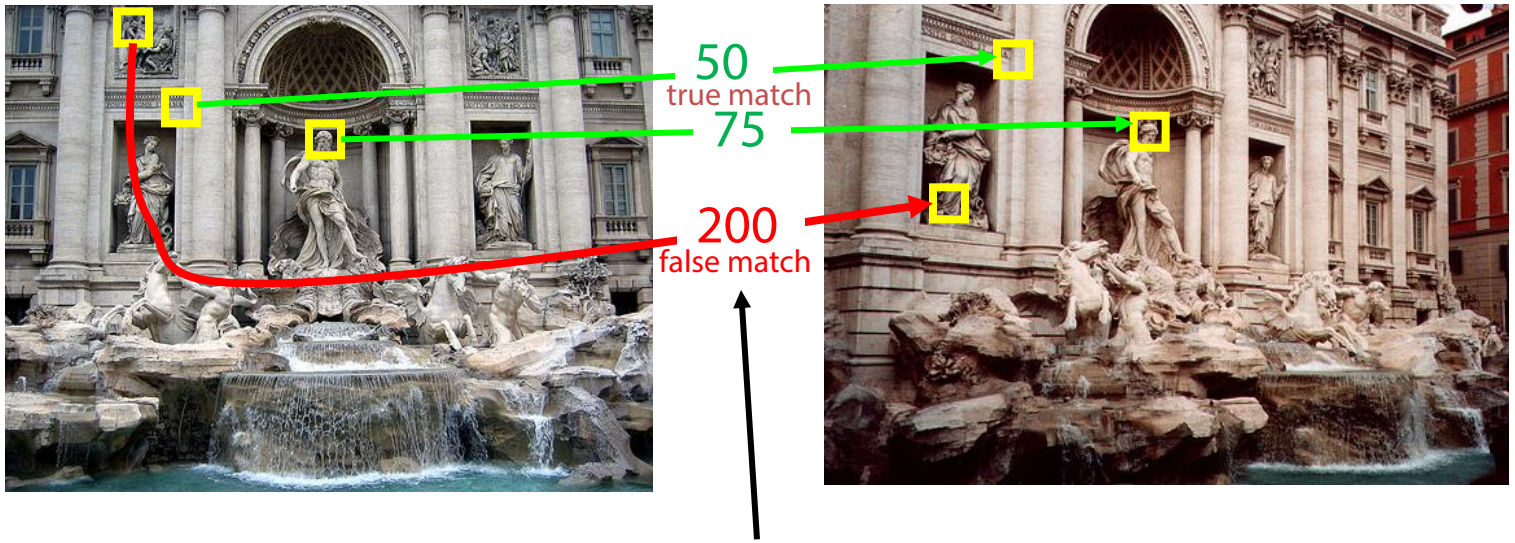
Evaluating the results

How can we measure the performance of a feature matcher?



True/false positives

How can we measure the performance of a feature matcher?



The distance threshold affects performance

- True positives = # of detected matches that survive the threshold that are correct
- False positives = # of detected matches that survive the threshold that are incorrect